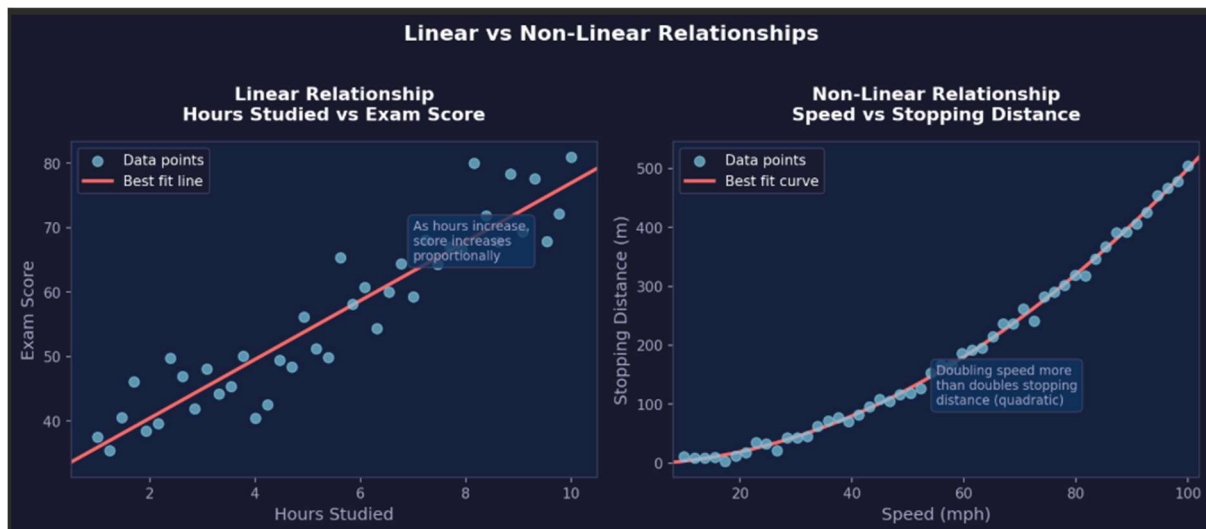


Summary of Machine Learning

When analysing a data set and looking to forecast outcomes based on features the best model to use is dependent on the properties of the data itself.

For basic linear relationships between 2 data sets where there is a linear relationship a linear regression will be the best way of modelling the data by finding a line of best fit.

Even for nonlinear relationships between 2 sets of data a polynomial regression will suffice and using machine learning for these would be excessive.



The point at which machine learning models becomes useful is when the relationship between the features and the outcome becomes too complex to be captured by a single equation.

This typically occurs when you have multiple interacting features whose combined effect on the outcome is non-linear and cannot be expressed as a simple formula. For example, predicting house prices from size, location, age, and condition simultaneously, where each feature influences the others in ways that no polynomial equation cleanly describes. At this point, models like Random Forests or Gradient Boosting become valuable because they can discover these interactions automatically from the data without you needing to specify the relationship in advance.

Neural networks take this further still, becoming the right choice when the data has inherent structure, such as the spatial relationships between pixels in an image, or the sequential dependencies between words in a sentence, that requires the model to learn hierarchical representations rather than just feature interactions. In these cases, the complexity of the pattern is so high, and the number of features so large, that hand-crafting any equation is not just impractical but impossible.

Machine Learning Models and their applications

Machine Learning Frameworks:

Machine Learning models can be trained in different manners based on their input data and the format of the output you are looking for.

Supervised Learning: Training a model where the training data has labelled output data. E.g. cancer cells where the feature data is the composition of the cell and the labelled output is if the cell is malignant or benign. The model is trained on hundreds/thousands of samples to recognise trends in the data and then tested on “Test datasets” to evaluate the accuracy of its predictions.

Unsupervised learning: We are not provided with any labelled output in the training data, so the model instead looks for structure in the data on its own. This might mean grouping data points into clusters with shared properties (e.g. K-means) or finding a smaller set of underlying patterns that explain the data's variance (e.g. PCA). For these models you have no known outcomes to generate accuracy scores in the way you do for supervised learning but can score the model on how well defined the clusters are and their distance from other clusters.

Self-Supervised learning: For some models like RNN when they are being trained on learning patterns in a body of text, like the Shakespeare RNN example, you are taking a window of text with a number of letters/words in it and then the output is the next word/letter directly after that sequence. The model has an output each time to train on, but it's not being labelled in the same way that supervised learning data sets do.

Reinforcement Learning: Learning through trial and error via reward signals from an environment, not fixed labels.

While it's possible to use all of the models for different purposes they have advantages for specific data sets.

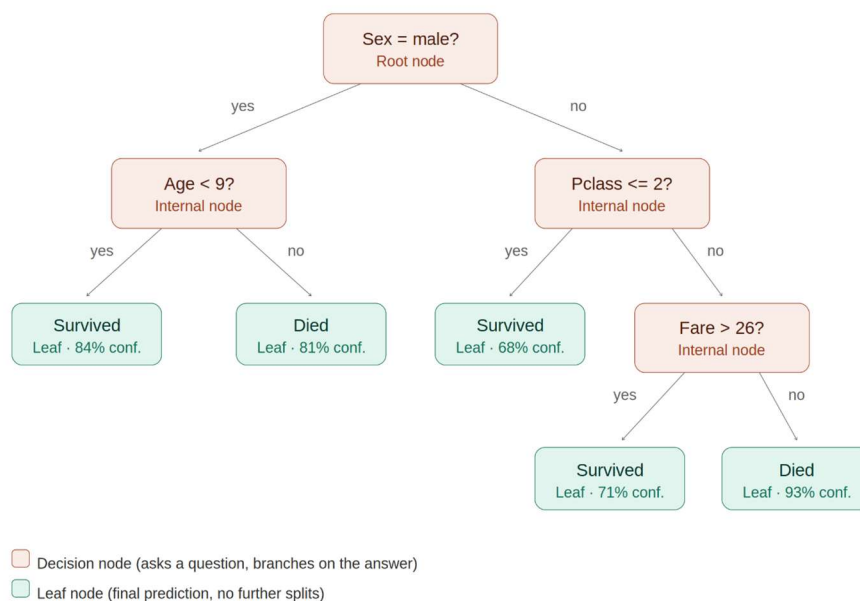
Model	Supervised/ Unsupervised	Best use for it	Examples
Random Forest	Supervised Learning only.	Structured tabular data, where the sequence of events isn't important. Better at smaller nosier data samples than Gradient Boosting. Random Forest is more resistant to overfitting.	Credit Scoring Fraud Detection The Titanic data set for predicting survivors. It can predict numerical and classifier outcomes.
Gradient Boosting	Supervised Learning only.	Structured tabular data. It uses a similar method to Random Forest. It can result in having a greater predictive accuracy but also takes more effort fine tuning the hyperparameters to get there.	Credit Scoring Fraud Detection The Titanic data set for predicting survivors. It can predict numerical and classifier outcomes.
K-means/ Hierarchical clustering	Unsupervised Learning only.	Groups by distance/similarity alone, no labels. The data sets will be tabular data not sequences. K-means is where you know how many clusters you expect, hierarchical clustering is where you don't.	Customer segmentation, anomaly/fraud flagging.
Recurrent Neural Network (RNN)		Targeted at modelling sequential data where the order of the sequence is important. Loads the elements one at a time.	Character/word level text generation, time series forecasting, simple sequence modelling
Convolutional Neural Network (CNN)		Spatial data, or sequences where local patterns matter more than long range order.	Image classification, object detection, image segmentation.

Transformer		Attention-based. This means the model loads all elements of the sequence at once (in parallel, not one-at-a-time) and computes how relevant every element is to every other element, based on their content.	Text generation, question answering, image classification
Autoencoder		Reconstruction based. The encoder compresses the input down into a smaller latent vector then the decoder attempts to rebuild something as close to the original input as possible and compares the reconstruction loss when training.	Anomaly detection, dimensionality reduction, image denoising, generative penetrating
Graph Neural Network (GNN)		Graph based. Built for data naturally structured as a graph instead of a sequence or grid	Social network analysis, molecule/drug property prediction, transaction fraud detection.
Support Vector Machine		Margin based, rather than find the best line that separates 2 classes it looks to maximize the distance between the decision boundary and nearest data point for each class.	Spam/text classification, small dataset image classification, handwriting recognition.

Decision Tree Models

Decision Tree models, such as Random Forests, used for predicting outcomes for tabular data where the sequence of the data isn't important, like the Titanic data set in the example in the Github. The titanic data set is using the data of the passengers such as their age, wealth, gender etc to predict if they would have survived the titanic or died. This is a simple data set used as an example for training models.

Below is an example of a decision tree for the Titanic data set.



The model aims to split the data using decision nodes that split the data into groups where the majority of the result is one of the outcomes, which is called its purity level. A perfectly pure node only contains one class (outcome), the higher the purity the better in terms of decision outcomes. The result of each branch does not need to have the same next question splitting the data, as seen below, it's selected independently at each instance to achieve the best grouping.

Gini – Impurity: Gini impurity measures how well-mixed the classes are at a node. A score of zero means all samples belong to the same class; higher values indicate a more mixed, impure node:

$$\text{Gini} = 1 - \sum_k p_k^2$$

For example, a node with 80 samples: 60 "Yes" and 20 "No":

$$\text{Gini} = 1 - \left(\left(\frac{60}{80} \right)^2 + \left(\frac{20}{80} \right)^2 \right) = 1 - (0.5625 + 0.0625) = 0.375$$

At each decision node the model goes through each decision available to calculate the one with the lowest weighted Gini Impurity of the 2 resulting nodes.

The weighted impurity accounts for the size of each child node:

$$\text{Weighted Gini} = \frac{n_{\text{left}}}{n} \times \text{Gini}_{\text{left}} + \frac{n_{\text{right}}}{n} \times \text{Gini}_{\text{right}}$$

The end of each branch, in green, is called a leaf and this is the end of the branch where the model is satisfied it's grouped the data sufficiently with decisions that it's capable of predicting an outcome with new features.

Ensemble Methods:

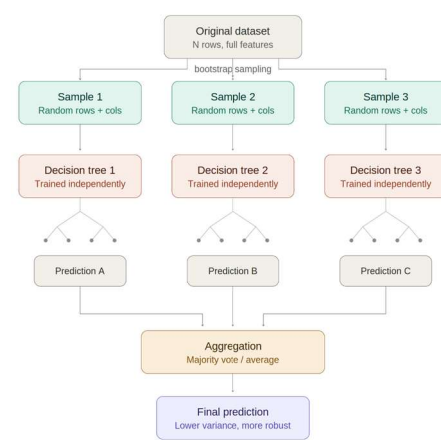
Decision trees can lack robustness, due to reasons like overfitting where a decision tree model is too granular in its leaves and then doesn't work very effectively for other data sets. The models can also be impacted by small changes in the training data that can lead to drastically different predictions, this is known as high variance, which means it's overly sensitive to its training data.

Ensemble methods help to tackle overfitting by aggregating the predictions of multiple classifiers and average them to improve the model's prediction accuracy.

The main 2 Ensemble methods are called Bagging (Bootstrap Aggregation) and Boosting.

Bagging:

A key method for reducing the probability of overfitting in a model is to use a method called bagging that takes multiple bootstrap samples, which are random subsets drawn with replacement (so the same row can turn up more than once in a sample) and creates trees from them. The predictions from these trees are either averaged or a majority vote is taken to get the final prediction.



For classification models like the one for Titanic a majority vote would decide the outcome, if 2 out of the 3 trees had “survived”, survived would be the prediction.

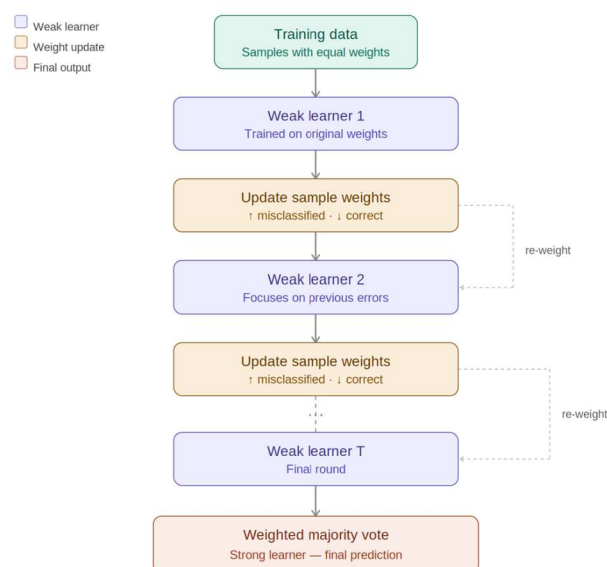
For regression models where you're calculating a number it would use the average of the predictions.

Out of Bag Error Estimation: Roughly 37% of the original data never gets selected for any given bootstrap sample. This left-out data is called the Out-of-Bag (OOB) sample, and it can be used as a free built-in validation set without needing a separate split.

Random Forests: Random Forests are one of the most widely used bagging methods and improve over traditional models by using a process known as Decorrelation. Decorrelation is the process of reducing the similarity between the models in the ensemble to ensure they don't all make the same mistakes. This increases the diversity of the model and improves the final prediction's reliability.

Random Forests act like regular bagging models with multiple decision trees but instead of allowing all features to be considered at every split, randomly select a subset of features for each tree at each split. This makes each tree more independent from each other. The average result from these trees has a much lower variance than if the trees were highly correlated, improving accuracy.

Boosting: This method builds trees sequentially, with each tree correcting the mistakes of the previous one. Boosting aims to create a strong model by focusing on instances the tree struggled to predict earlier.



The process begins by training the first model on the entire dataset, and then the misclassified or poorly predicted answers are given higher importance, with their weights being increased. The next model in the sequence is trained to correct these mistakes. Correctly classified samples get their weights reduced. The process continues for T rounds, and the final answer is the weighted average across all T learners where better performing learners carry more influence.

Boosting is very effective in reducing bias and can be applied to both classification and regression problems. Examples of boosting models are AdaBoost, XGBoost and Gradient Boosting.

Hyperparameters:

The parameters used for running the model are called hyperparameters and fine tuning these can improve the models performance by doing things like reducing overfitting:

A) The minimum sample splits: When a branch gets to a minimum number of samples it stops making new decision branches and the last results become the leaves regardless of the percentage split between outcomes.

B) The maximum number of levels. Prevents the model from going on too long.

C) The maximum features. Limits the number of features a model is allowed to consider.

D) Maximum samples – This controls the number of samples in each tree. The less samples in each tree the more trees the result is averaged across.

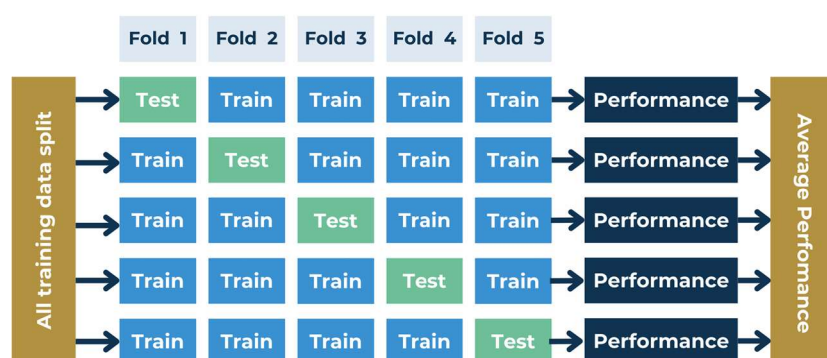
E) Learning Rate – This is how quickly a model learns from its mistakes. Too low and it won't learn fast enough, too big and it might skip important details.

You can use software libraries to find the optimum hyperparameter configuration for you.

Cross Validation is a popular method for optimizing performance where it tests the model on different pieces of data. This helps ensure the model works well on the unseen data as well as training data.

GridSearchCV:

Cross validation splits the data into many parts, called folds, so that every data point gets a chance to be used for both testing a training.



GridSearchCV builds on this to tune hyperparameters, by running different combinations of hyperparameters to find the best settings for the models. You set the range of hyperparameters you want to test it on, and it iterates through combinations finding the best result using cross validation.

Neural Networks Summary

Artificial Neural Networks:

Artificial Neural networks were inspired by the neurons in the human brain where neurons receive messages via synapses from multiple sources. The receiving neuron accumulates these incoming signals and if the combined signal crosses a threshold value it fires sending a signal to the next neuron.

Each of the messages coming into the neuron can have different "weights" in terms of importance. This inspired the ANN concept of using layers of "hidden layer" neurons where each neuron applies a weight to the figure it receives from the other multiple neurons and passes the weighted sum of those inputs via an activation function, which introduces a non-linear transformation of the output. The weights are set with random amounts initially but through training the model the weights are updated so the model learns to become more accurate in predicting the output.

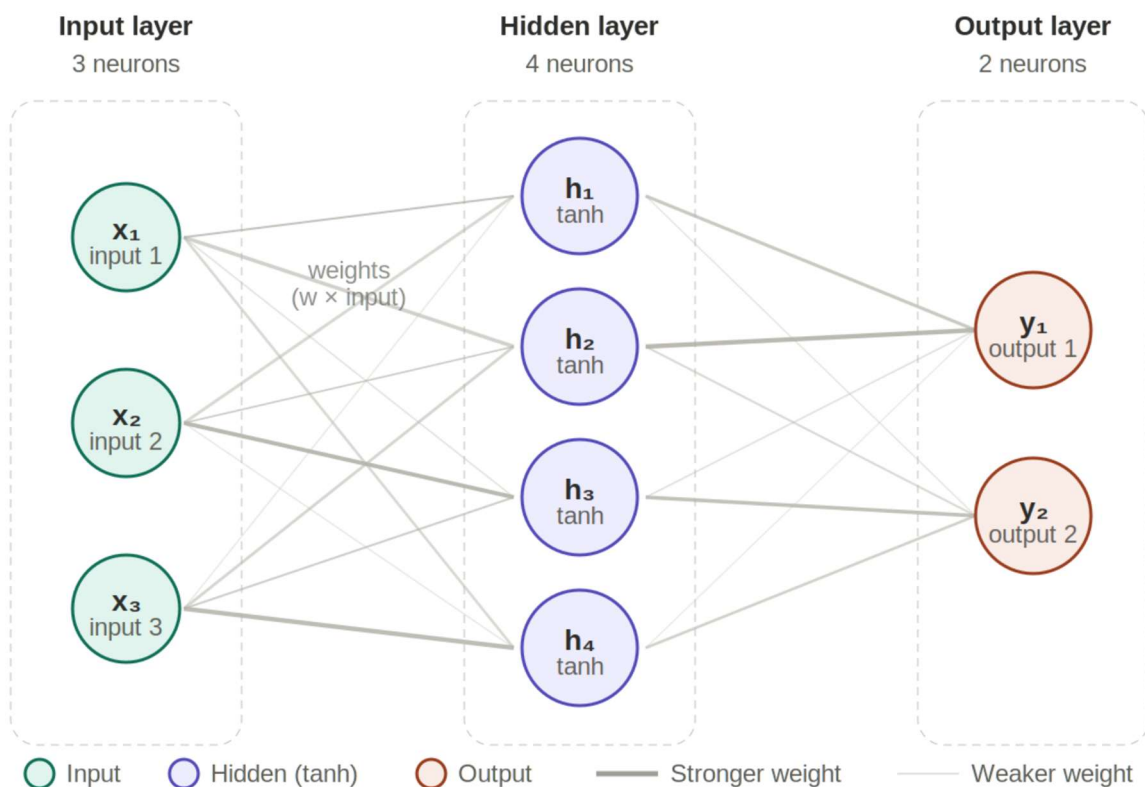


Diagram generated by Claude.

Multi-Layer Perceptron:

The above diagram is for an MLP which is a specific type of Artificial Neural Network that:

- i) Feeds forwards only, from left to right and doesn't loop back
- ii) Has every neuron connected to every other neuron
- iii) Has multiple layers, including an input, hidden (can have multiple), and output layer.

Input Layer:

This is defined by how many features we are training the model on.

Features are data sets that the model trains on to predict the output. If we're using the order of words in sentences for a body of literature for training, the number of input neurons will equal the size of the input window, e.g. if we feed the model the previous 5 words to predict the next, there will be 5 input neurons.

If we're training it to predict the movement of a financial instrument over time, we may want to also train the model on other variables that we know contribute to the price movement in that asset. E.g. for gold we can train the model on gold price movement in conjunction with currency, inflation, gold ETF level movements etc, where each one of those variables would be a feature that would have their own input neuron.

Hidden Layer:

The number of hidden layer neurons is determined by the data scientist creating the model and there is no defined rule on how many neurons are required and the outcome will be found by trial and error. There are python libraries we can use to automate the process of finding the optimal parameters to run the model on.

Too few hidden layer neurons, and you risk underfitting the model and not training it sufficiently to generate an accurate output.

Too many hidden layer neurons, and you risk overfitting the model, and it learns the noise in the data as well as the useful information, making it less capable of predicting when the data is asked to predict outputs using new data sets. An MLP with one hidden layer is called a Shallow Neural Network, a model with multiple layers is called a Deep Network.

Activation Functions:

At the end of each hidden layer is an activation function that takes the weighted sum of all the inputs and passes them through a function that produces a new number that can

be passed onto the next layer. In the above diagram each hidden layer neuron would be summing the weighted inputs from the 3 input neurons. The type of activation function used depends on the purpose of the MLP.

Sigmoid Function:

The function compresses outputs into a range between 0 and 1, making it a useful model for when binary outcomes are predicted, e.g. like models used to assess if a cancer tumour is benign or malignant. The output is a probability score where if the score exceeds 0.5 the tumour would be classified as malignant and less than 0.5 as benign.

Hyperbolic Tangent Function (tanh):

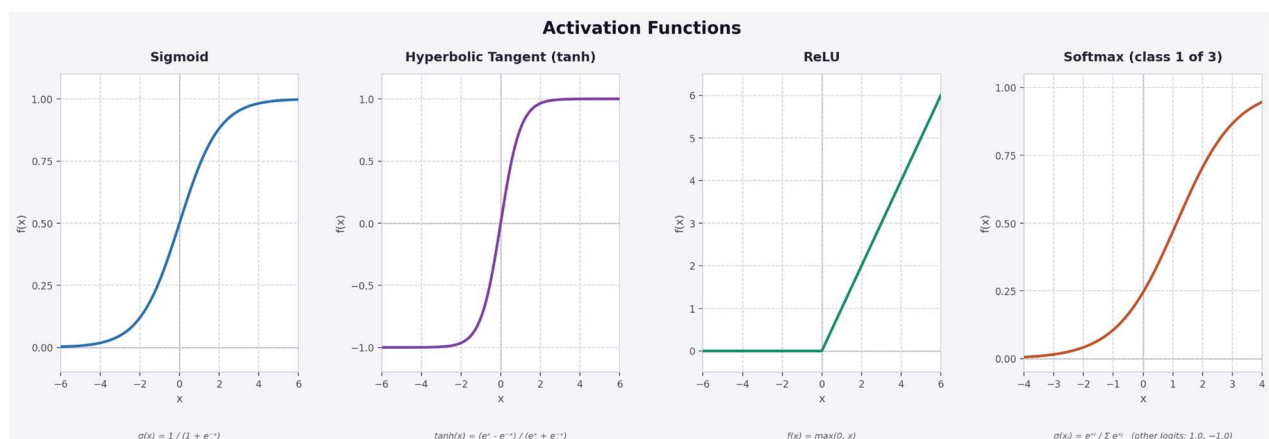
Generates output values between -1 and 1. This works well for when a model needs to handle negative values, such as in stock market prediction.

Rectified Linear Unit (ReLU):

Outputs the input directly if it's positive, otherwise outputs a zero. This is preferred for deep networks, especially in applications like image recognition as it's effective at mitigating the vanishing gradient problem.

Softmax Function:

This is used in multi-class classification problems where the network needs to assign probabilities to different classes ensuring the sum of the probabilities is 1. E.g. if we're building a network to identify animals in pictures. Each animal will have an output neuron, and this will assign to each output a probability it's that outcome.



Output Layer:

This depends on what output you're trying to generate with the model. If you're looking for an output of a number or a single result, then it will have one output neuron. If you've created a classification model where you're looking for an output of "True" or "False" for

example or for the model to be able to predict if a picture has an animal defined in the training process, you will have output neurons for each of those outcomes.

Training the model:

Forward Propagation

For an MLP the input data passes through the input layer into the hidden layers with their values having weights applied to them as they enter the hidden neuron. These weights are assigned randomly at the beginning and are updated through the training process. The sum of the weighted inputs then has a bias figure added to them and this bias figure is initially set at close to zero. The addition of a bias value allows the activation function to shift left or right, which can be important for fine-tuning the model and allowing it to fit the data better. For the above example the sum of the values at h1 would be:

$$y = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$$

If b is a large positive it means the neuron can activate even with small x values and if it's negative it will dampen large x values impact.

This ability to shift activation thresholds helps neural networks better fit the data by adjusting how inputs are combined and processed. The models pass the summed figure through the activation function at the end of each hidden layer, either into the next hidden layer if there are multiple hidden layers or into the output layer.

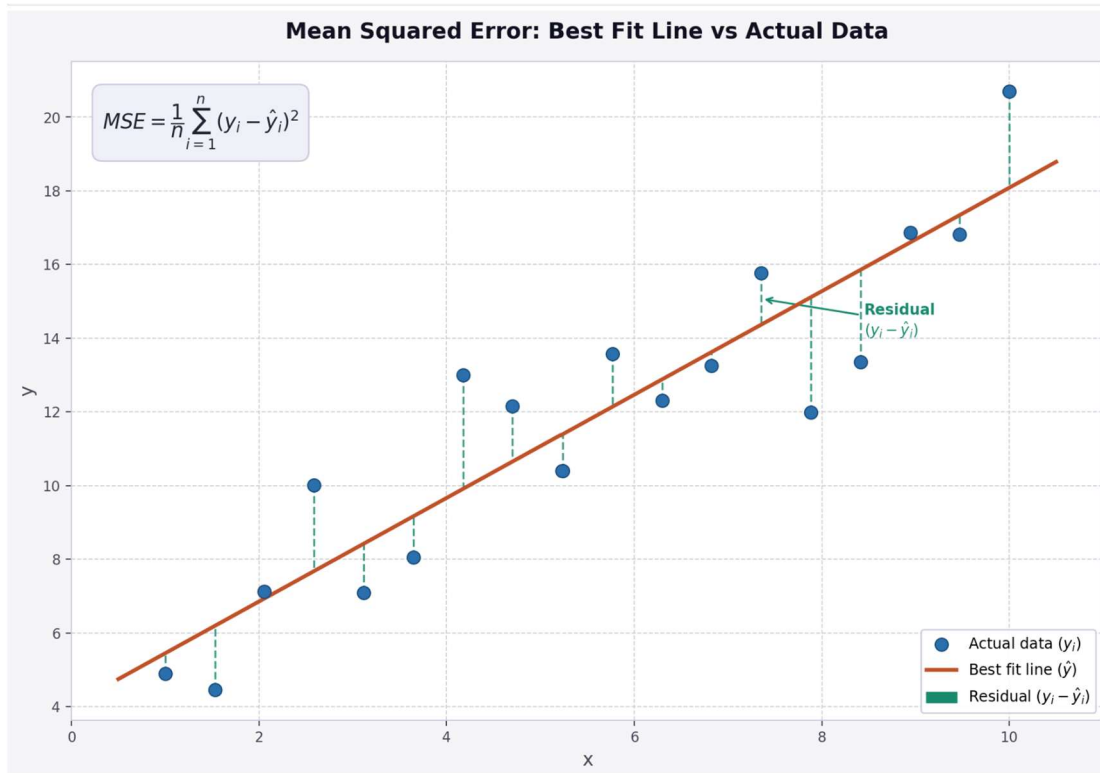
Once the figures get to the output layer to generate a prediction result. In model training this prediction can then be compared to the actual outcome and we can calculate the error value, which is the difference between the actual and predicted figure. This forward pass through the model is called Forward Propagation.

Computing the loss:

Different metrics are used for different model types.

Mean Squared Error:

MSE or RMSE is when the output is a continuous number, like predicting a house price or temperature. For modelling a data set with one feature you can chart a scatter plot with the data and apply a line of best fit in the data. For this simple example the Mean Square error is the average of the squared difference between the error and the actual answer.



Binary Cross Entropy:

This is used for evaluating the performance of binary outcome models, e.g. ones that have "True/False" outcomes. The model will normally have a sigmoid output activation.

Categorical Cross-Entropy:

This is used for multi-class classification, e.g. for more than 2 possible outcomes, such as predicting the next word in a sentence or identifying animals in pictures. Cross-entropy measures how different the predicted probability distribution is from the true distribution and is more effective for MSE when there is more than one output.

Back Propagation and updating the weights:

The weights for each neuron begin as being a random small figure. The way the model trains to improve its performance and reduce the loss function is through back propagation. Back Propagation is based on the principle of Gradient Descent, where the aim is to minimize the loss function.

Gradient of loss function calculated:

First step is to calculate the gradient of the loss function, which indicates the direction and rate at which the weights need to be adjusted to minimise the loss.

Local Gradients are computed:

For each neuron, local gradients are computed. These are partial derivatives of the neurons output with respect to it's inputs, which are then used to work out the impact of each weight on the loss. Using partial derivatives, we hold all other weights and biases constant while making small changes to one to assess its impact on the loss. This helps us analyse how every individual weight impacts the total error.

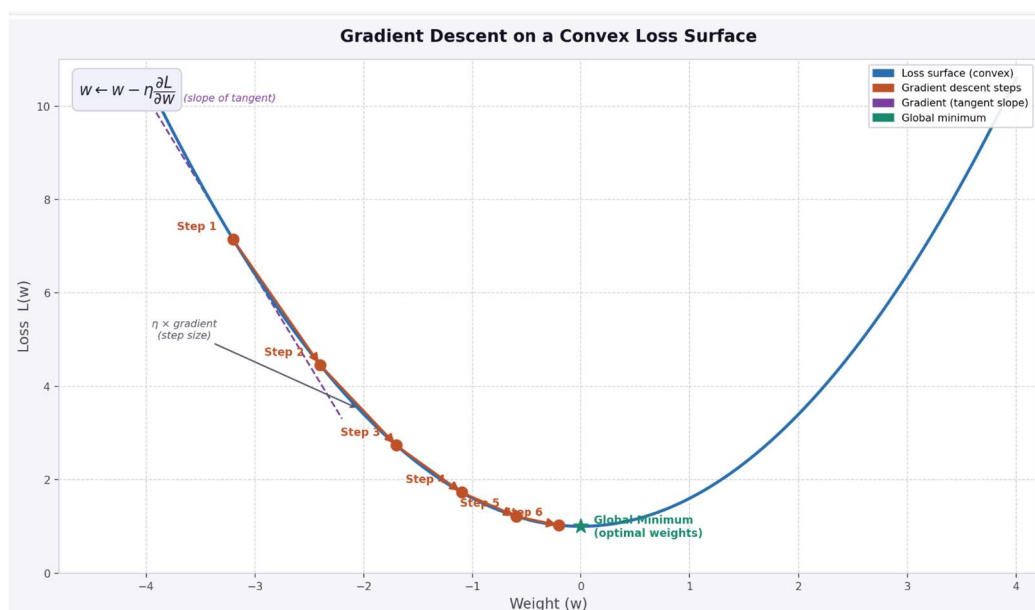
The error is distributed to all weights in the network proportional to their contribution to the total error.

Updating the weights and biases:

The final step uses the calculated gradients to update the weights and biases. The size the weights are moved is a product of the learning rate, η which is defined by the user when setting up the model and the gradient. The gradient is large on the steep sides of the curve and zero at the bottom.

$$\text{step size} = \eta \times \frac{\partial L}{\partial w}$$

Below you can see the curve that is the change in loss relative to the weights. The objective of the process is to move down the curve to a progressively lower gradient each time until you find the optimal weight at the bottom where the rate of change is closest to zero. This point should be the minimal loss value you can achieve that will give the best output results.



The data for a training session can be fed into the model in batches rather than feeding the whole data set as a vector in one go. The benefit for this is it's better for hardware efficiency doing lots of small runs and by running backward propagations after each mini batch you get a more stable less erratic convergence.

An epoch is one complete pass through the entire training dataset. Forward propagation, loss calculation, and backpropagation, repeated across all the batches until every training example has been seen once. You can run as many epochs as you like, but training for too long causes the model to overfit. This is where the model starts memorising the training data rather than learning generalisable patterns. The optimum point to stop is when the validation loss stops improving, even if training loss is still falling. Stopping at this point, often called early stopping, gives the model the best chance of performing well on data other than the training set.

Validation Set: This is a data set that you hold back and don't train the model on but use it periodically to generate outputs to calculate a loss figure. This is called the validation loss. It will fall in line with the main loss calculation for the first few epochs but when overfitting kicks in the validation loss may increase.



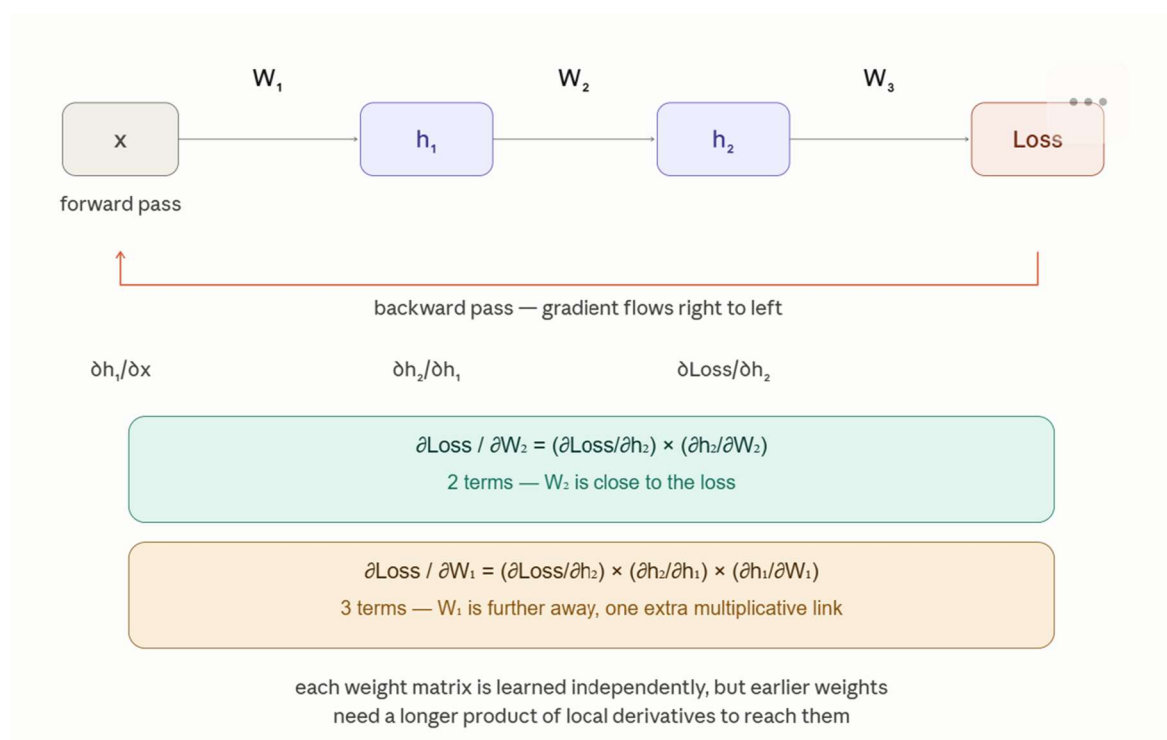
Vanishing Gradient Problem:

If we're using the sigmoid or tanh activation function for a model with not many hidden layers it works fine. Once the number of layers starts to get larger we incur the vanishing gradient problem.

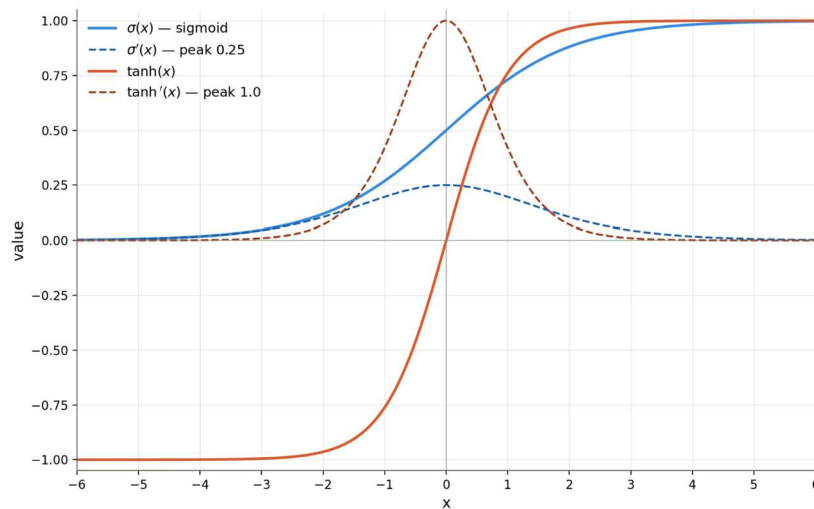
For back propagation a loss value at the end of a network depends on the final layer's weights, but that final layer is impacted by the layers before it.

To know how much to adjust the weights in earlier layers to change the final loss you need to track the influence through the entire path.

The Chain Rule does this as it gives you a way to multiply together a sequence of local derivatives to get an overall derivative across a long dependency chain.



The problem we have with using sigmoid or tanh activation functions is that their derivatives have small maximums so when you're multiplying a lot of these together if you have many rows it converges to zero. Sigmoid has a maximum derivative value of 0.25 and tanh is 1. The problem is worse for sigmoid than tanh, but tanh still can be impacted.



As per the above example the back propagation technique uses the chain rule to work backwards through the hidden layers updating the weights. The more layers it has to go through to get to the last layer the more gradients it's multiplied by and if they are small that means the smaller the amount the weight is going to be adjusted by which limits the learning capabilities of the model.

ReLU doesn't shrink the gradient in the same way as its derivative is either 1 (for positive inputs) or 0, so an active neuron passes the gradient through unchanged.

ReLU's derivative is:

- 0 if input < 0
- 1 if input > 0

The one issue it has is that if the neuron input becomes negative its derivative will be zero, which is called the dying ReLU problem. Leaky ReLU where instead of going to zero it has a negative gradient have been created to solve for this.

Recurrent Neural Networks

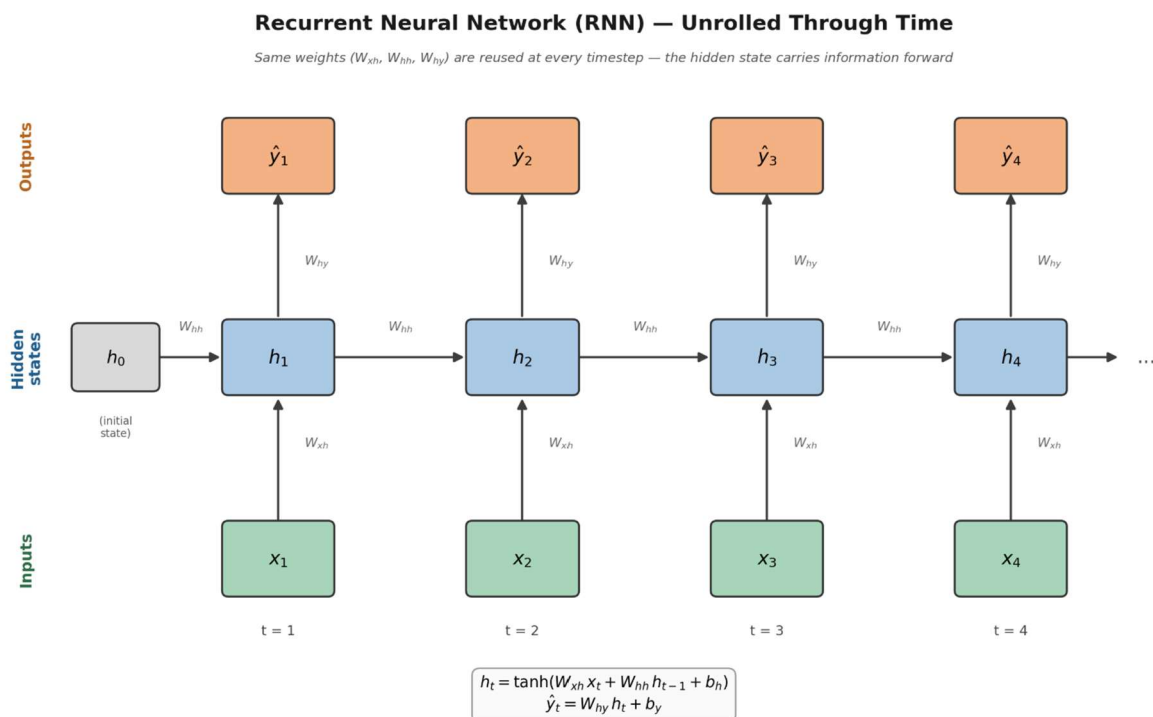
RNN's or Recurrent Neural Network are used to model sequential data where the order of the sequence is important for accurately determining the outcome. Other ANN models (Artificial Neural Networks) treat each input independently, whereas RNN's feeds the output from each step into the next one as it deems the order of the sequence to be important not just the values in it. The hidden layer in an RNN captures all prior history not just the last n-1 words.

RNN's can be used for:

Time Series Forecasting: Stock prices, weather etc

Text Generation: The model can be trained on a body of text data such as the complete works of Shakespeare and then generate text in that style, although modern NLP's now use Transformers for this purpose.

Sentiment Analysis: Determining the sentiment of a sentence based on the words and structure of the sentence.

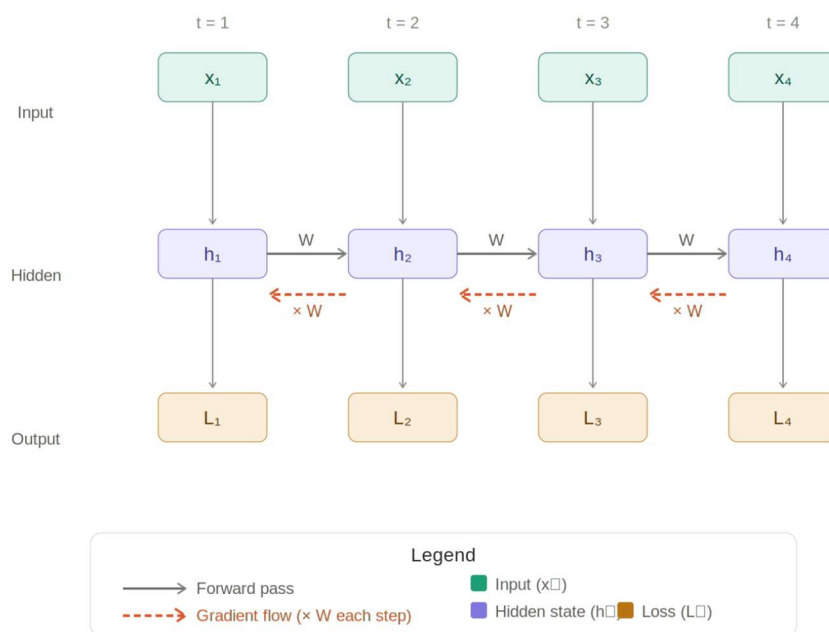


In the above RNN words are fed one by one into a chain of identical neural networks, so for “The cat sat on the mat”, x_1 = “The”, x_2 = “cat” etc. Each word will be converted into a numerical value using word embedding. For training on a set of text, you create an integer index by applying one hot encoding to each word and making this list of unique words and their encoding into a matrix.

Each network produces 2 outputs, 1 to the output layer and one to the next hidden layer.

Training Process:

Back propagation can be used to train models, but for RNN's they use a special version called Back Propagation Through Time. This technique allows the propagation of error gradients back through time across the recurrent connections, helping the model retain the information in its hidden state. An RNN processes a sequence by reusing the same weights at every time step, so it's the same network repeated T times.

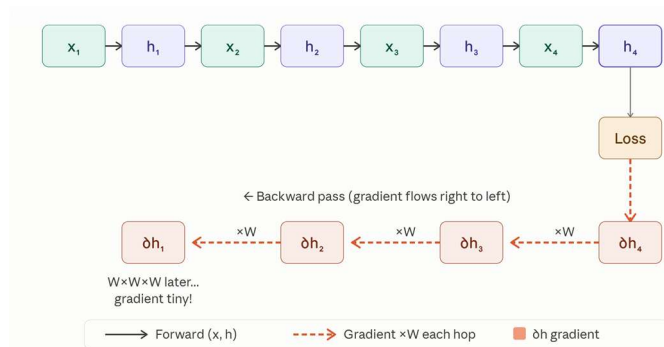


To apply back propagation, you unroll the RNN into the below chain, treating each time step as a separate layer. Then standard back propagation runs backwards through the whole unrolled chain. The total gradient for W is the sum across all time steps:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial W} \text{ at } t=1 + \frac{\partial L}{\partial W} \text{ at } t=2 + \frac{\partial L}{\partial W} \text{ at } t=3 + \frac{\partial L}{\partial W} \text{ at } t=4$$

Then a single weight update is applied to W using that summed gradient:

$$W = W - \text{learning rate} \times (\text{sum of all gradients})$$



The more layers you back propagate through the greater chance of having disappearing gradient or if the values get too big an “exploding gradient”.

To resolve for exploding gradients you can use gradient clipping where it ensures it doesn’t exceed a preset threshold every time it’s used on each layer.

For vanishing gradients 2 different methods are used for RNN’s to mitigate this risk, they are called LSTMs and GRUs.

Long Short-Term Memory (LSTM):

LSTM’s tackle the issue of vanishing gradients by incorporating a complex mechanism of gates that control the flow of information.

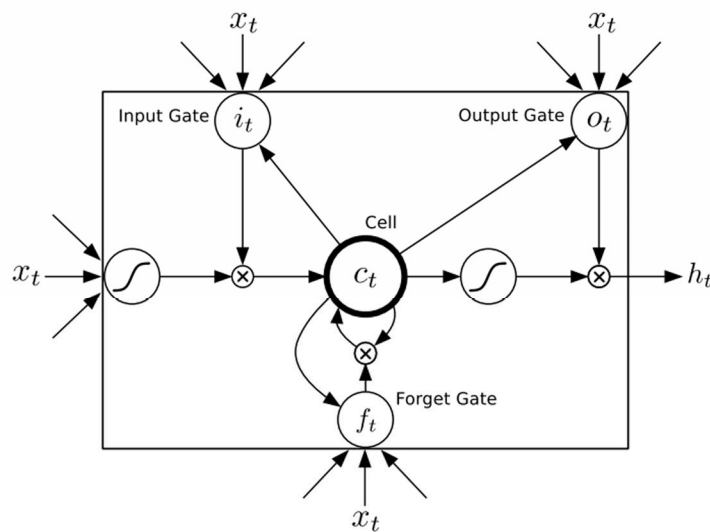


Figure 2: LSTM Cell (Graves, 2014).

An LSTM unit typically comprises of a cell containing 3 gates:

i) **Forget gate:** This determine the information from the previous cell state that should be forgotten. It uses the previous hidden state and current input combined and passed through a sigmoid function, the result of this sigmoid function is shown below.

$$f_t = \sigma(W_x f \cdot x_t + W_h f \cdot h_{t-1} + W_c f \cdot c_{t-1} + b_f)$$

It then multiplies the output from the previous layer by function result:

$$f_t \times c_{t-1}$$

If the sigmoid function is close to zero it “forgets” the previous layers output and if it’s near 1 it retains it.

ii) **Input gate:** The input gate decides what new information should be allowed into the cell. It uses a sigmoid function to decide which values to update. This determines how much of the new candidate information should be written.

$$i_t = \sigma(W_x i \cdot x_t + W_h i \cdot h_{t-1} + W_c i \cdot c_{t-1} + b_i)$$

A tanh function creates a vector of new candidate values that could be added to the state. This determines the candidate new values we could write.

$$\tilde{C}_t = \tanh(W_{xc} \cdot x_t + W_{hc} \cdot h_{t-1} + bc)$$

These are then multiplied together $i_t \times \tilde{C}_t$.

Finally, the cell state is updated by multiplying the previous cell state by the forget gate scale and adding it to the new candidate values (scaled by the input gate).

$$c_t = (f_t \times c_{t-1}) + (i_t \times \tilde{C}_t)$$

Output Gate: The output gate is the final part which decides what the next hidden state should be. It uses the updated cell state and squashes it through a sigmoid function along with the previous hidden layer value and current input.

$$o_t = \sigma(W_{xo} \cdot x_t + W_{ho} \cdot h_{t-1} + W_{co} \cdot c_t + bo)$$

The updated current state is then passed through a tanh function and scaled by the output gate to give you the new hidden state value. The new hidden state will always have a value between -1 and 1 given that it's a sigmoid output times a tanh output.

$$h_t = o_t \times \tanh(c_t)$$

Gated Recurrent Units (GRU's):

GRU's are a streamlined version of the LSTM model. GRU handles similar issues to the LSTM model, it's streamlining generally means it can run faster but it has fewer learned weight parameters which means it can struggle to learn more complex patterns.

The core differences between the LSTM and GRU model is:

i) There is no cell state, h_{t-1} is the only recurrent signal flowing through, doing the job both h and c did in the LSTM.

ii) GRU has 2 gates instead of 3:

A) The Reset gate: Similar to the forget gate, controls how much of the past hidden state to use when computing new candidate content.

B) The Update gate: Combines what the LSTM's forget and input gates did separately into one gate. It decides simultaneously how much old memory to keep and how much new information to write in.

The table below summarizes the pros and cons of using them for different scenarios:

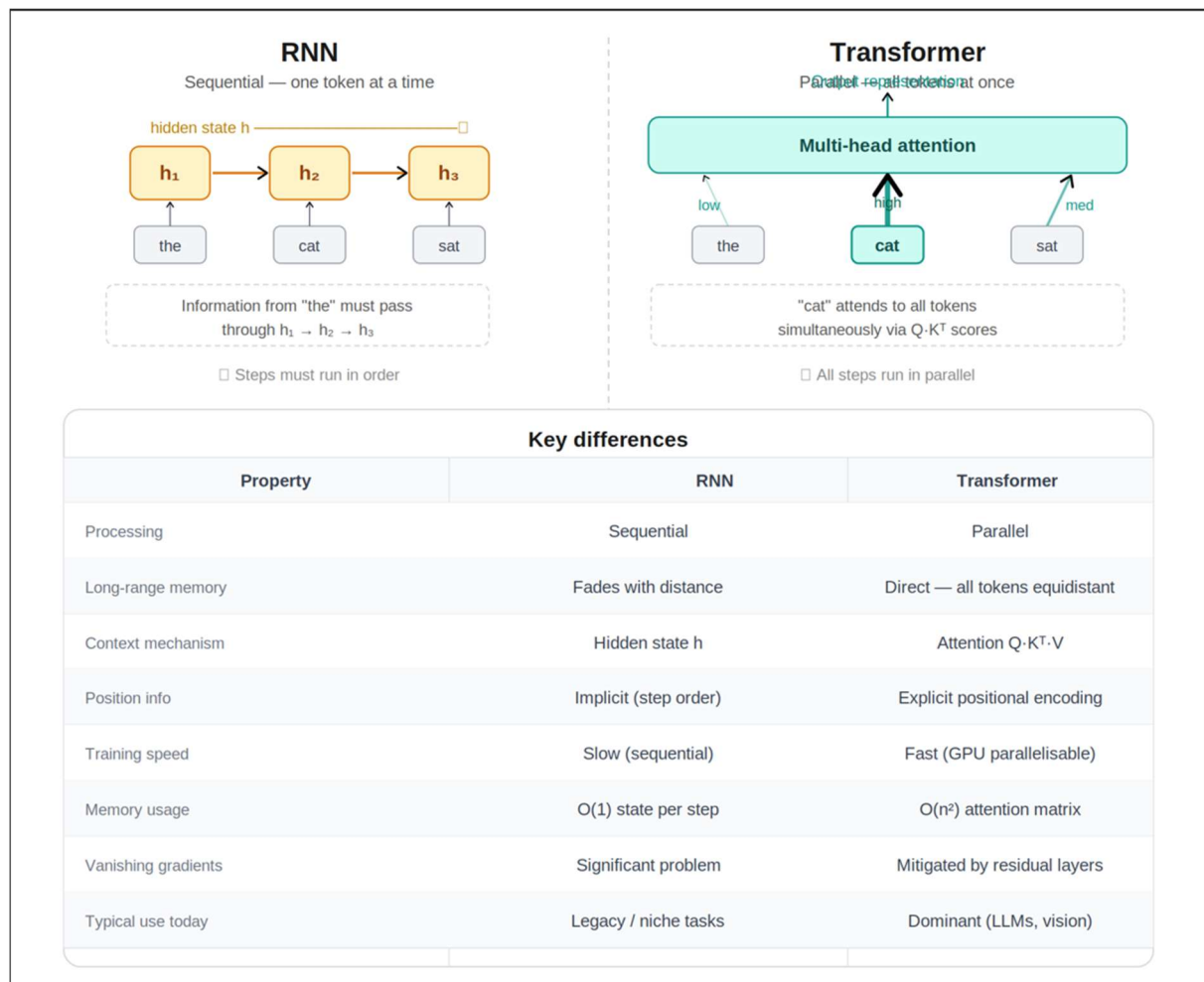
Scenario	LSTM	GRU
Long sequences e.g. long documents	✓ Better Cell state retains	Can struggle No cell state
Small dataset limited training data	Can overfit More parameters	✓ Better Fewer parameters
Speed critical e.g. real-time inference	Slower 3 gates, cell state	✓ Better 2 gates, simpler
Complex language e.g. text generation	✓ Better More expressive	Workable Less fine control
Time series e.g. stock prices, sensors	Workable Often overkill	✓ Better Efficient, accurate
Needs fine tuning maximise performance	✓ Better More knobs to turn	Less flexible Gates are fused
Speech / audio e.g. ASR, music	✓ Better Handles long context	Workable Simpler patterns ok
Quick prototype testing an idea fast	Heavier More to configure	✓ Better Fast to get running

Transformers

RNN's were popular initially for text prediction but had their limitations, the discovery and implementation of transformers is what led to the current advances in LLM's that we see today in the form of Claude, Gemini and ChatGPT etc.

RNN's read one token at a time and go from left to right carrying a hidden state vector forward like a running summary. The issue that this had was that the early tokens carried less prominence and became "forgotten". LSTMs helped with this issue but not sufficiently to make this an effective solution.

Transformers load all the tokens from a data set in parallel simultaneously, so they all carry the same weight and the earlier words from the sequence are not forgotten, unlike in RNNs where the vanishing gradient problem causes early tokens to lose influence. The tokens look at each other to look at where they are relative to each other and these computes weighted attention scores.



The Attention Score:

The transformer learns by adjusting three sets of things via gradient descent:

i) The weight matrices – the learned projections that turn the input vector into Q, K and V. They determine how attention scores are computed using the below equation:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Q = Query: “What am I looking for?” – a vector derived from the current token via a learned weight matrix W_q

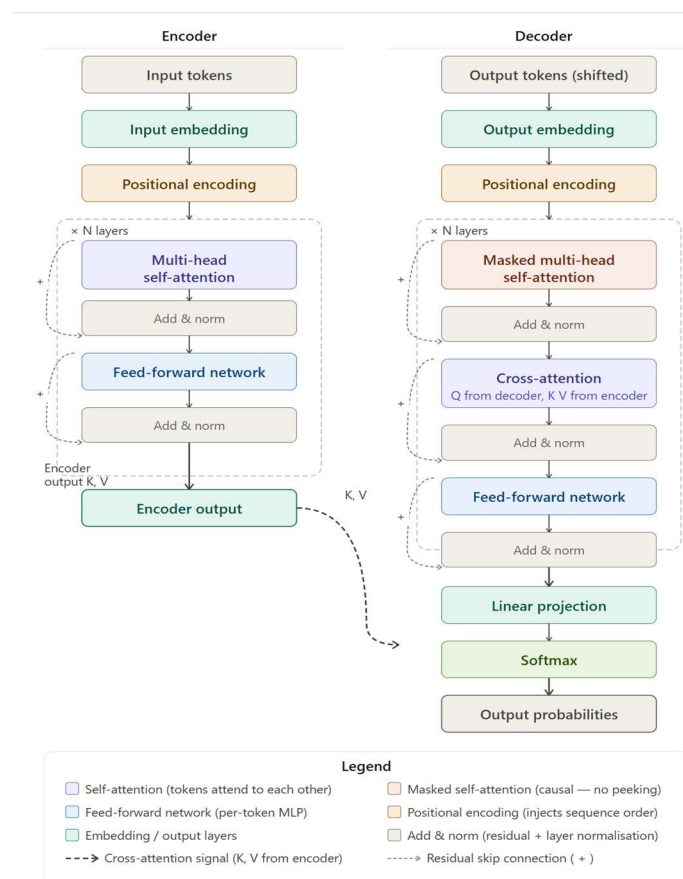
K = Key: “What do I contain?” – a vector derived from every token via a learned weight matrix W_k

V = Value: “What information do I offer?” – a vector derived from every token via a learned weight matrix W_v

QK^T = Dot product between the query and all keys – producing the relevance score between the current token and every other token.

$\sqrt{d_k}$ = Scaling factor between

Transformer workflow:



1. Input embedding + positional encoding

Each token (word or subword) is converted to an embedding vector. Because the transformer processes all tokens in parallel it has no inherent sense of order, so a positional encoding is added to each embedding to inject sequence position information.

2. Encoder stack (repeated N times)

Each encoder layer has two sub-layers:

- Multi-head self-attention: Every token attends to every other token, building up a rich contextual representation. "Multi-head" means this happens in parallel across several attention heads, each learning different relationships
- Feed-forward network: A simple MLP applied independently to each token's representation.

After each sub-layer there is a residual connection (adds the input back to the output) and layer normalisation. These together stabilise training.

3. Decoder stack (repeated N times, in encoder-decoder models)

Each decoder layer has three sub-layers:

- Masked self-attention: Same as encoder self-attention but tokens can only attend to previous tokens, not future ones.
- Cross-attention: The decoder attends to the encoder's output, connecting source and target sequences.
- Feed-forward network: This is the same as the encoder

4. Output projection + softmax

The final decoder output is projected onto the vocabulary size and passed through a softmax function to produce a probability distribution over the next token.